

TOOLS IN ACTION · DEVOXX BELGIUM 2026

Tap into the NVIDIA ecosystem from Java

with TornadoVM's Hybrid API. No JNI, no C++.

Michalis Papadimitriou

University of Manchester - Neo4j

Mary Xekalaki

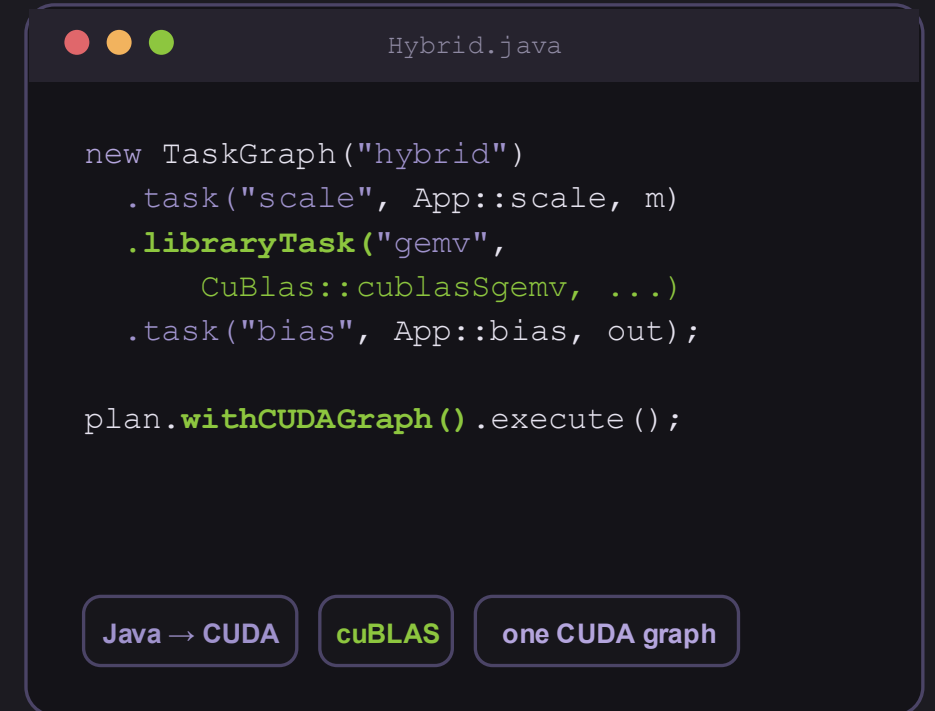
University of Manchester - Beestream Ltd

Beehive Lab · The University of Manchester

DEVOXX™
BELGIUM

Antwerp
Tuesday 6th of October 2026

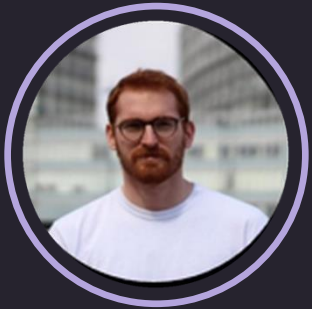
tornadovm.org



Java kernels and NVIDIA libraries, one task graph

About us

TornadoVM contributors, working on Java for GPUs.



Michalis Papadimitriou

@mikepapadim

Senior Software Engineer · Neo4j
Research Fellow · University of Manchester

TornadoVM Contributor

PREVIOUSLY

ML Compiler Engineer · OctoAI (acquired by NVIDIA)
Research Software Engineer · Huawei Research Labs

michail.papadimitriou@manchester.ac.uk



Mary Xekalaki

@mairooni

Principal Software Engineer · Beestream Ltd
Research Associate · University of Manchester

mary.xekalaki@beestream.ai

Java runs the enterprise.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.

WHERE JAVA ALREADY IS

Spring · Quarkus · Micronaut

Kafka · Flink · Spark

LangChain4j · Spring AI

Neo4j · Cassandra · Elasticsearch

Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.

WHERE JAVA ALREADY IS

Spring · Quarkus · Micronaut

Kafka · Flink · Spark

LangChain4j · Spring AI

Neo4j · Cassandra · Elasticsearch

WHERE THE SPEED IS

cuBLAS · cuBLASLt · CUTLASS

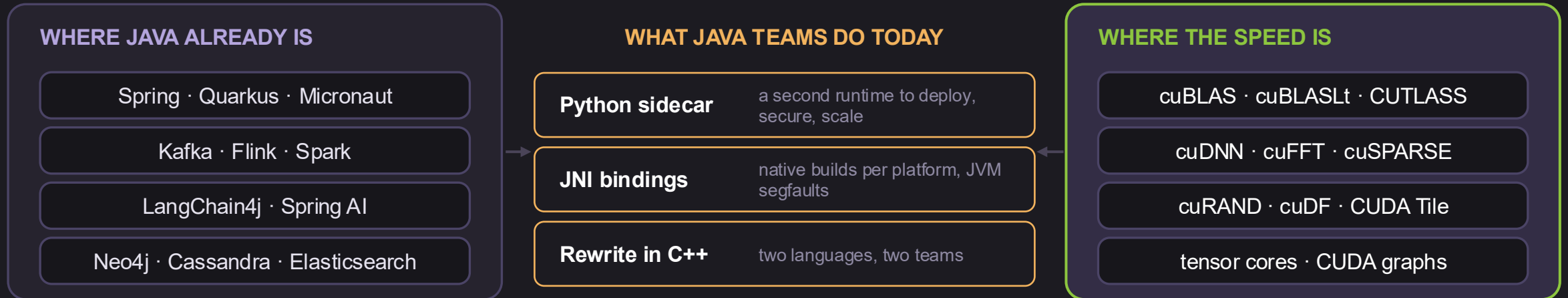
cuDNN · cuFFT · cuSPARSE

cuRAND · cuDF · CUDA Tile

tensor cores · CUDA graphs

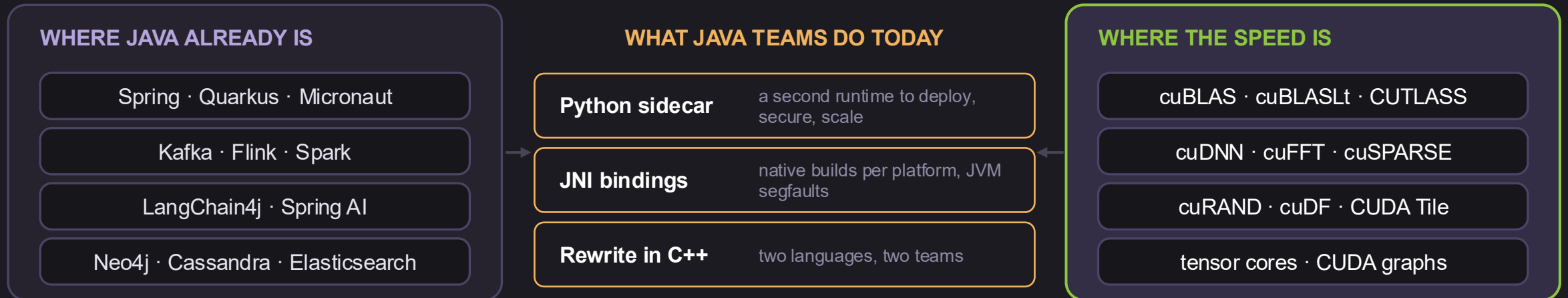
Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.



Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.



TornadoVM Hybrid API NVIDIA libraries as a Maven dependency, called from Java, on the JDK you already run.

Why the hassle? NVIDIA already tuned them

Years of per-GPU tuning, one dependency away.

cuBLAS

Dense linear algebra

Use it for ML inference, embedding search, risk models

Saves you a tuned GEMM per GPU generation

`Sgemv · Sgemm · GemmEx · batched`

cuBLASLt

Fused matmul

Use it for transformer layers: matmul + bias + GELU

Saves you writing fused epilogue kernels

`FP32 / FP16 / FP8 · bias + GELU`

cuFFT

Fourier transforms

Use it for signal and audio, spectra, time series

Saves you tuning FFTs for every size

`C2C · R2C · C2R · batched · 2D`

cuDNN

Deep-learning primitives

Use it for CNNs, activations, LLM attention

Saves you conv and attention kernels per GPU

`conv2d · softmax · pool · SDPA`

cuSPARSE

Sparse matrices

Use it for graphs (PageRank = SpMV), recommenders

Saves you balancing irregular sparse rows

`CSR SpMV · SpMM`

CUTLASS

Fused GEMMs

Use it for GEMM + bias + activation, native speed

Saves you C++ template metaprogramming

`S/H/B-GEMM · fused epilogues`

cuDF

GPU DataFrames

Use it for ETL, joins, group-by, CSV / Parquet reads

Saves you columnar kernels for relational ops

`readers · filter · join · group-by`

THE CATCH

How do you call them from Java?

Today: a FFM/JNI shim per library, per platform, and a host round trip per call.

01

TornadoVM

An overview

A Java framework that JIT-compiles selected methods to GPU kernels and manages the device around them.

The TornadoVM ecosystem

Any JDK from 21 to 27, three GPU backends, and the tools and integrations around them.

JDK 21–27 DISTRIBUTIONS

OpenJDK	Microsoft JDK
GraalVM	Azul Zulu
Eclipse Temurin	SapMachine
Red Hat Mandrel	

runs on any JDK



compiles per GPU

HARDWARE BACKENDS



OpenCL

AMD · Intel · NVIDIA



CUDA

NVIDIA



Metal

Apple Silicon

DEVELOPER TOOLS

-  TornadoVMPulse
-  TornadoViz
-  TornadoInsight
-  NVIDIA Nsight Systems

DOCKER IMAGES



NVIDIA and Intel images

DISTRIBUTION



Maven Central · SDKMAN!

INTEGRATIONS



Spring Boot



Quarkus



LangChain4j



Flink

Active since 2018 · 30+ contributors · 30+ research papers · used by the European Space Agency (ESA)

TornadoVM in a nutshell (*hint: is not a VM*)

Runs on the JDK you already have. Not a JVM, not a binding layer.

Your Java application

ordinary bytecode, still running on the JVM

Tornado API

@Parallel · KernelContext · TileContext · TaskGraph · TornadoExecutionPlan

Runtime

task graph · transfer tracking · device memory · CUDA graphs · streams · NVTX

JIT compiler

bytecode → Tornado IR → CUDA C (NVRTC) or CUDA Tile C++ (tileiras) → cubin

CUDA backend

SIMT + CUDA Tile

OpenCL

AMD · Intel · NVIDIA

Metal

Apple Silicon

Library providers

cuBLAS · cuFFT · cuDNN · ...

Gives you 4 ways to program **NVIDIA** GPUs

From least to most control, plus the vendor's own kernels. Mix them freely in one graph.

01

easiest start

@Parallel

Annotate a loop; iterations become GPU threads.

```
void add(FloatArray a,
        FloatArray b,
        FloatArray c) {
    for (@Parallel int i = 0;
        i < c.getSize(); i++)
        c.set(i, a.get(i)
            + b.get(i));
}
```

02

full control

KernelContext

The CUDA thread model: ids, shared memory, barriers.

```
void sum(KernelContext ctx,
        FloatArray a,
        FloatArray out) {
    int t = ctx.localIdx;
    float[] s = ctx
        .allocateFloatLocalArray(256);
    s[t] = a.get(ctx.globalIdx);
    ctx.localBarrier();
    for (int h=128; h>0; h/=2) {
        if (t < h) s[t] += s[t + h];
        ctx.localBarrier();
    }
    if (t == 0)
        out.set(ctx.groupIdx, s[0]);
}
```

03

NEW · CUDA Tile

TileContext

Describe one tile block; the tile compiler does the rest.

```
void mm(TileContext tc, ...) {
    var av = tc.partition(
        tc.view(a, m, k), TM, TK);
    var bv = tc.partition(
        tc.view(b, k, n), TK, TN);
    int r = tc.bidX();
    int c = tc.bidY();
    Tile acc=tc.zeros(F32,TM,TN);
    for (int s=0; s<k/TK; s++)
        acc = tc.mma(av.load(r,s),
            bv.load(s,c), acc);
    cv.store(acc, r, c);
}
```

04

NEW · cuBlas · ...

libraryTask

NVIDIA's kernel as a node, on the same buffers.

```
new TaskGraph("g")
    .transferToDevice(
        EVERY_EXECUTION, m, v)
    .task("scale",
        App::scale, m)
    .libraryTask("sgemv",
        CuBlas::cublasSgemv,
        OP_T, rows, cols, 1f,
        m, lda, v, 1, 0f, out, 1)
    .task("bias",
        App::bias, out)
    .transferToHost(
        EVERY_EXECUTION, out);
```



02

The Hybrid API

your kernels + NVIDIA's libraries

Java kernels, CUDA Tile kernels and vendor-optimised libraries in one task graph, on shared GPU buffers.

No more hassle: one Java shape for all

The same libraries, now a `.libraryTask` away, on your Java kernels' device buffers.

cuBLAS

Dense linear algebra

Use it for ML inference, embedding search, risk models

Saves you a tuned GEMM per GPU generation

`Sgemv · Sgemm · GemmEx · batched`

cuBLASLt

Fused matmul

Use it for transformer layers: matmul + bias + GELU

Saves you writing fused epilogue kernels

`FP32 / FP16 / FP8 · bias + GELU`

cuFFT

Fourier transforms

Use it for signal and audio, spectra, time series

Saves you tuning FFTs for every size

`C2C · R2C · C2R · batched · 2D`

cuDNN

Deep-learning primitives

Use it for CNNs, activations, LLM attention

Saves you conv and attention kernels per GPU

`conv2d · softmax · pool · SDPA`

cuSPARSE

Sparse matrices

Use it for graphs (PageRank = SpMV), recommenders

Saves you balancing irregular sparse rows

`CSR SpMV · SpMM`

CUTLASS

Fused GEMMs

Use it for GEMM + bias + activation, native speed

Saves you C++ template metaprogramming

`S/H/B-GEMM · fused epilogues`

cuDF

GPU DataFrames

Use it for ETL, joins, group-by, CSV / Parquet reads

Saves you columnar kernels for relational ops

`readers · filter · join · group-by`

THE ANSWER · ONE CALL SHAPE

```
.libraryTask(  
    name,  
    Provider::method,  
    args...)
```

Same device buffers as your Java kernels. No JNI. Also on Maven Central: `cuRAND`.

Anatomy of a libraryTask: cuFFT in a task graph

Every argument of the cuFFT call, then the whole HybridLive graph, line by line.

ONE LIBRARY CALL

[cuFFT API reference](https://docs.nvidia.com/cuda/cufft) → docs.nvidia.com/cuda/cufft

```
.libraryTask("forward", CuFft::cufftForwardR2C, signal, spectrum, N, 1)
```

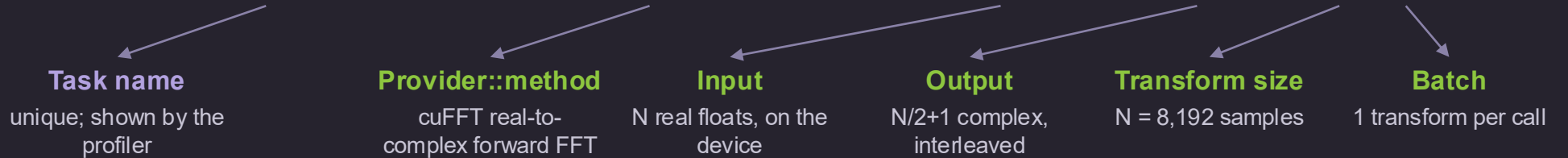
Anatomy of a libraryTask: cuFFT in a task graph

Every argument of the cuFFT call, then the whole HybridLive graph, line by line.

ONE LIBRARY CALL

[cuFFT API reference](https://docs.nvidia.com/cuda/cufft) → docs.nvidia.com/cuda/cufft

```
.libraryTask("forward", CuFft::cufftForwardR2C, signal, spectrum, N, 1)
```

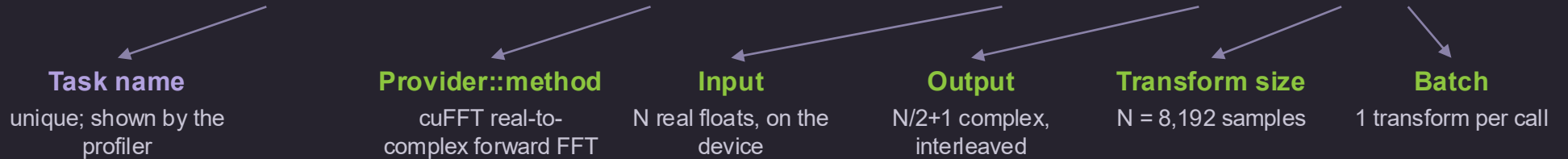


Chain of libraryTasks + JIT in a task graph

ONE LIBRARY CALL

[cuFFT API reference](https://docs.nvidia.com/cuda/cufft) → docs.nvidia.com/cuda/cufft

```
.libraryTask("forward", CuFft::cufftForwardR2C, signal, spectrum, N, 1)
```



THE WHOLE GRAPH, LINE BY LINE

```
var tg = new TaskGraph("hybrid") .....▶ Named graph; nothing runs yet
.transferToDevice(EVERY_EXECUTION, signal, cutoff) .....▶ Inputs to the GPU, every run
.libraryTask("forward", CuFft::cufftForwardR2C, ...) .....▶ cuFFT: signal → spectrum
.task("lowPass", HybridLive::lowPass, spectrum, cutoff, BINS) .....▶ Java kernel: zero bins ≥ cutoff
.libraryTask("inverse", CuFft::cufftInverseC2R, ...) .....▶ cuFFT: spectrum → signal
.task("normalize", HybridLive::scaleBy, filtered, 1.0f / N) .....▶ Java kernel: divide by N
.transferToHost(EVERY_EXECUTION, filtered, spectrum); .....▶ Results back, every run
```

CUDA runtime features also available from Java

Memory belongs to the task graph. CUDA's real mechanisms are typed, composable options on the plan.

EXECUTION PLAN OPTIONS

```
try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
    plan.withCUDAGraph() .....▶
    .withIntraPlanConcurrency() .....▶
    .withStagedTransfers() .....▶
    .withGridScheduler(grid) .....▶
    .execute(); .....▶
}
```

WHAT IT DOES

Captured once, replayed as one launch

Independent tasks run on parallel streams

Host→device copies via pinned buffers

You set grid and block size per task

Runs; the options are no-ops on OpenCL, Metal

IN CUDA

`cudaGraphLaunch`

`cudaStream pool`

`cudaMallocHost ring`

`<<<grid, block>>>`

MEMORY, DECLARED ON THE GRAPH

```
.transferToDevice(FIRST_EXECUTION, weights) .....▶
.persistOnDevice(kvCache) .....▶
.transferToHost(EVERY_EXECUTION, logits); .....▶
```

WHAT IT DOES

Copied on the first run only, then stays

Stays on the GPU, never copied back

Copied back after every run

IN CUDA

`cudaMemcpy, once`

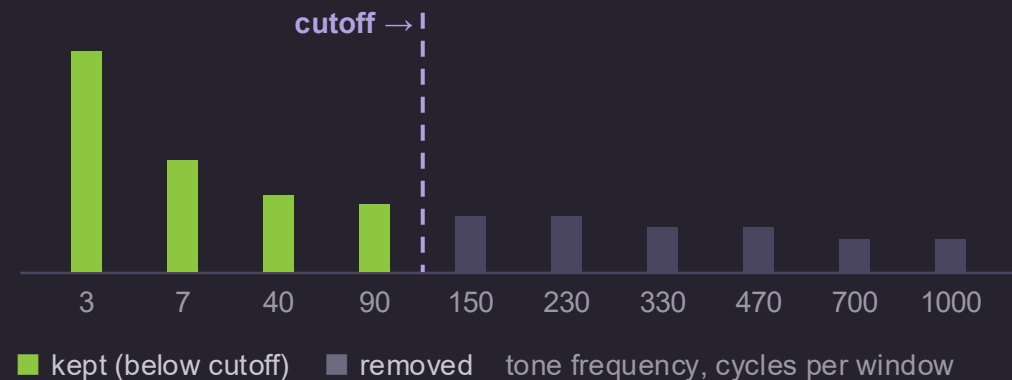
`device-resident`

`cudaMemcpy D→H`

The demo app: a live low-pass filter

HybridLive.java runs the graph ~10 times a second and proves every frame against the exact answer.

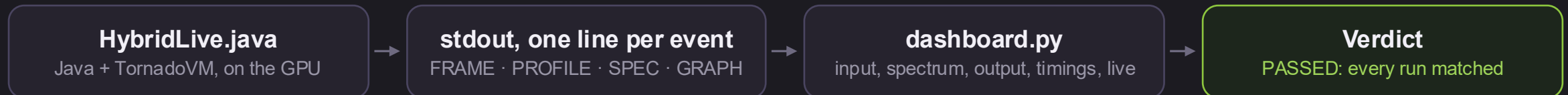
THE SIGNAL: 10 TONES, 8,192 SAMPLES



WHAT THE APP DOES

- 1 **Sweep** cutoff falls from 1,200 bins to 5 over 110 frames, one `execute()` per frame
- 2 **Check** each output vs the exact sum of the kept tones: max error < 0.01 is "correct"
- 3 **Profile** TornadoVM's profiler reports every task's GPU time and bytes copied
- 4 **Graph** 300 runs plain, 300 with `withCUDAGraph()`: median time of each, still checked

HOW IT'S WIRED



CUDA Tile from Java: TileContext

cuTile for Java, through TornadoVM

ONE NEW CONTEXT

TileContext

`tc.*` every tile op hangs off it

CUDA 13.3+

sm_80+

● You write

Tile shapes

$T_M \times T_N \times T_K$

Which tile this block owns

`tc.bidX(), tc.bidY()`

Math on whole tiles

`load · mma · store`

● Tile compiler decides

- Threads per block
- Thread indices
- Shared memory & barriers
- Tensor-core instructions

Java + TileContext

TornadoVM JIT

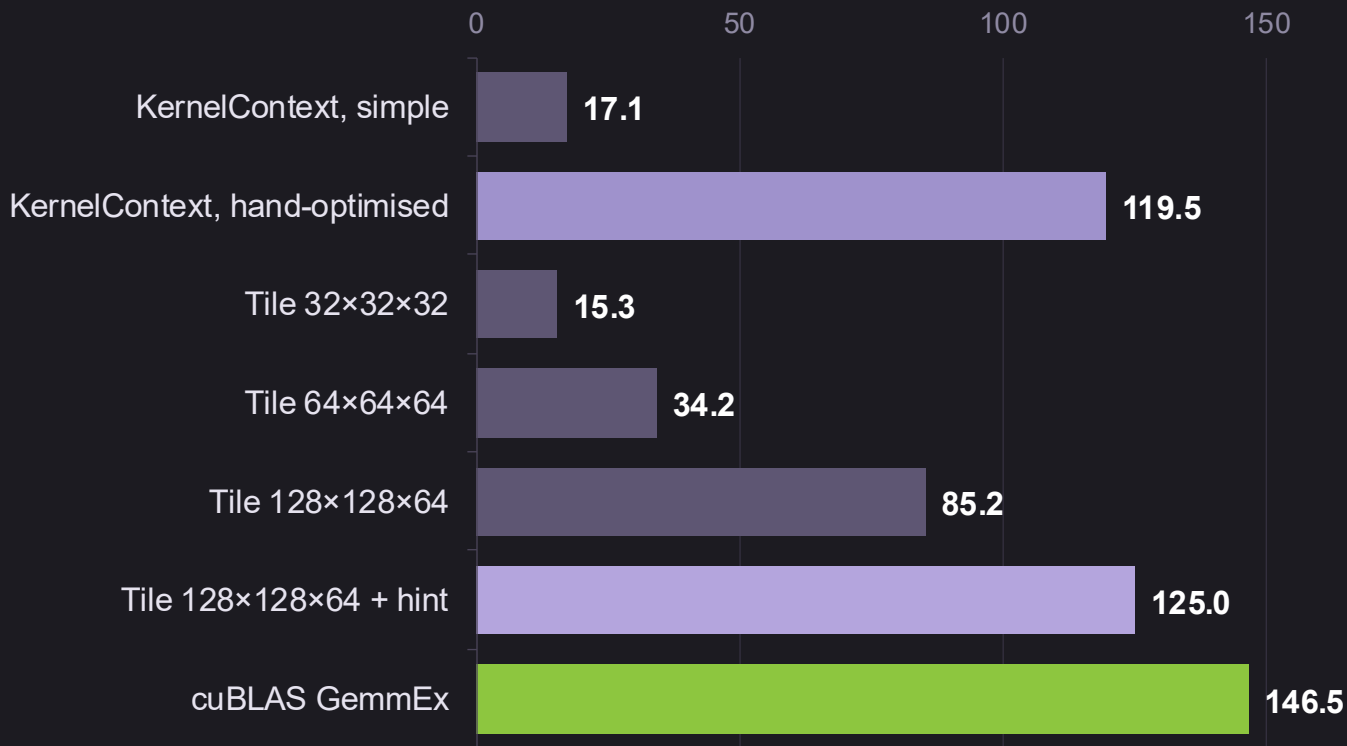
`cuTile · ct::*`

NVIDIA tile compiler

GPU tensor cores

Ten lines of tiles vs a hand-tuned kernel

FP16 GEMM, FP32 accumulate, n = 2048. Kernel time (nsys), RTX 4090, TornadoVM 7.0.0.



1.05×

faster than hand-optimised KernelContext

5.6×

from the tile shape alone, same kernel body

4.5×

faster than native CUDA Tile with a runtime n: the JIT knows the extents

TFLOP/s, higher is better

Use case 1

jitLLM

LLM inference in pure Java. Hybrid prefill with NVIDIA libraries, head to head with llama.cpp.

j ⚡ tLLM



Building LLM Inference Engines in Java with TornadoVM and the NVIDIA Ecosystem



Thanos Stratikopoulos (TornadoVM)



Michalis Papadimitriou (Neo4j)



Orion Papadakis (TornadoVM)

Java Language & Platform

Conference

INTERMEDIATE

More in our other talk on jitLLM internals!

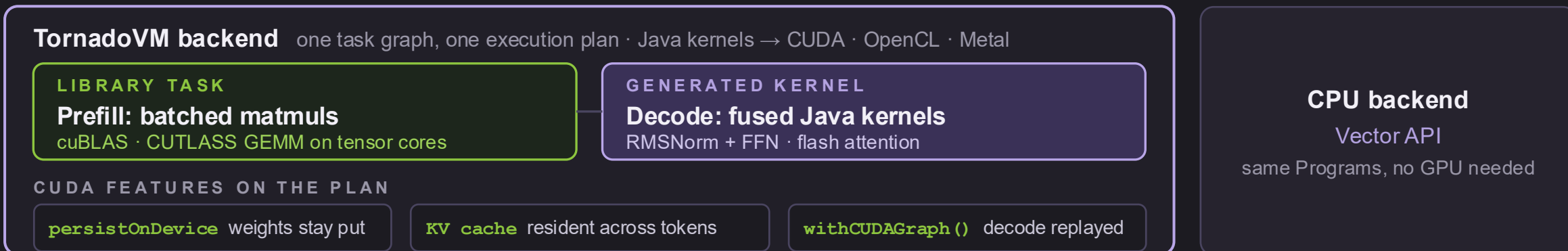
JitLLM: Java inference with **NVIDIA** libraries

Modern Java on top. Underneath, TornadoVM calls NVIDIA libraries and CUDA features.

depends on

Integrations	CLI · OpenAI server · LangChain4j · Quarkus · Spring AI
Public API	LocalModels · LocalModel · GenerationSession (13 stable types)
Engine	LLMEngine · Scheduler · KvCacheManager · BlockPool · PrefixCache
Models	GGUF → ModelProvider (ServiceLoader) → config + weights + tokenizer
Programs	backend-neutral ops: RMSNorm, RoPE, matvec, attention, SwiGLU, sampling
Runtime	DataType · tensor descriptors · memory planning · ExecutionPolicy

BACKEND SPI



jitLLM vs llama.cpp

Same GPU, same GGUF weights, llama.cpp at the same batch size. Five of sixteen configurations.

PREFILL · native libraries

1.12×

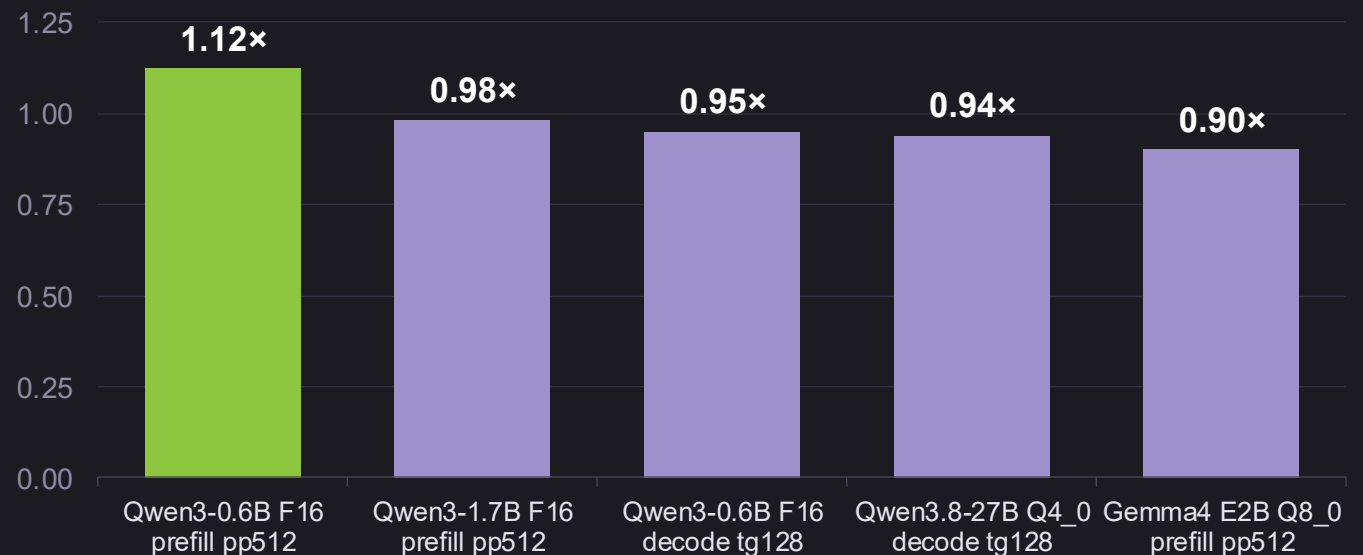
faster than llama.cpp

Qwen3-0.6B F16 · pp512 · batch 512

50,300 vs 44,805 tok/s

cuBLAS GEMMs for the prompt batch, Java kernels for everything else.

Throughput relative to llama.cpp = 1.00



tok/s	0.6B pp512	1.7B pp512	0.6B tg128	27B tg128	Gemma4 pp512
jitllm	50,300	21,296	443.3	40.5	8,477
llama.cpp	44,805	21,754	464.6	43.2	9,435
jitllm setup	native b512	native b512	JIT	JIT	native b128

Use case 2

Apache Flink on **NVIDIA** Libraries



GPU offload inside Flink's SQL planner with
TornadoVM: generated kernels and NVIDIA libraries
in one task graph.

 **From Install to Insight: A Hands-On GPU Lab for Java Developers**

 **Thanos Stratikopoulos** (TornadoVM)  **Christos Kotselidis** (University of Manchester)  **Mary Xekalaki** (University of Manchester)

[Java Language & Platform](#) [Hands-on Lab](#) [BEGINNER](#)

Hands-on lab to set up a local GPU-backed LLM in Java using GPULlama3.java, TornadoVM, and DevovxGenie. Participants ins...

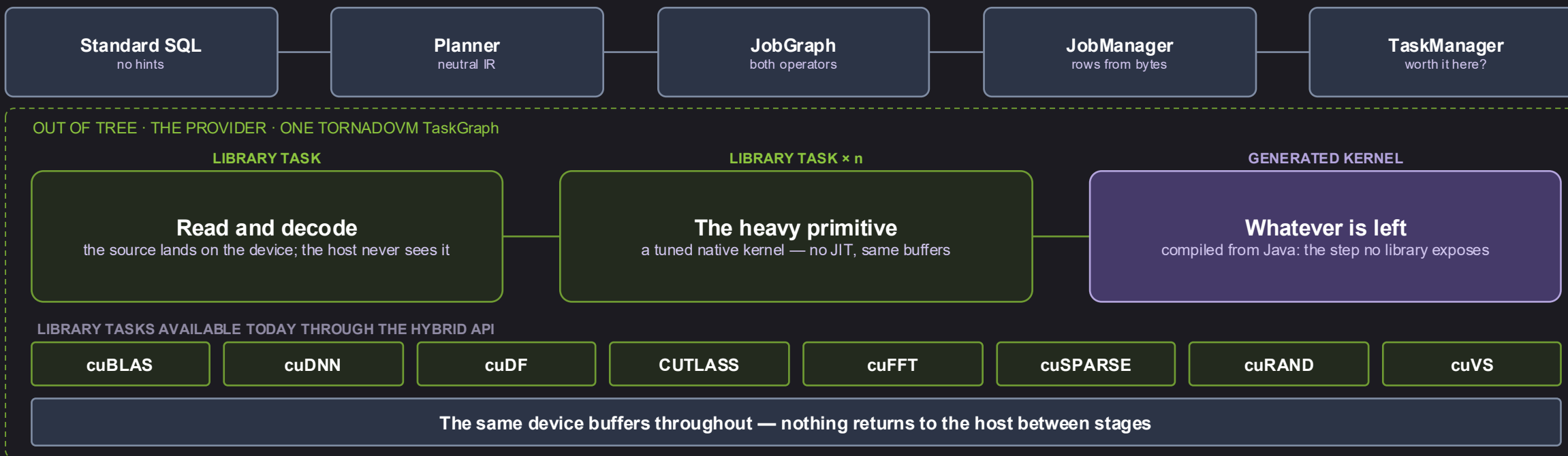
More in our other talk on TornadoVM internals!



Where the libraries plug in

One task graph, two kinds of node. A library task calls a tuned native kernel; a generated task is compiled from Java. They share the buffers.

APACHE FLINK · UNCHANGED ABOVE THIS LINE



The mix is per query: this talk's example reads Parquet with cuDF, matches k regexes with cuDF, and folds the masks with a generated kernel.

Everything above the line is Flink as it ships. Everything inside the panel is out of tree and loads only where a device is present.



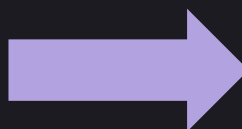
Screening logs: one SQL query, one task graph

You write plain SQL. Flink and TornadoVM turn it into one task graph on the GPU.

YOU WRITE

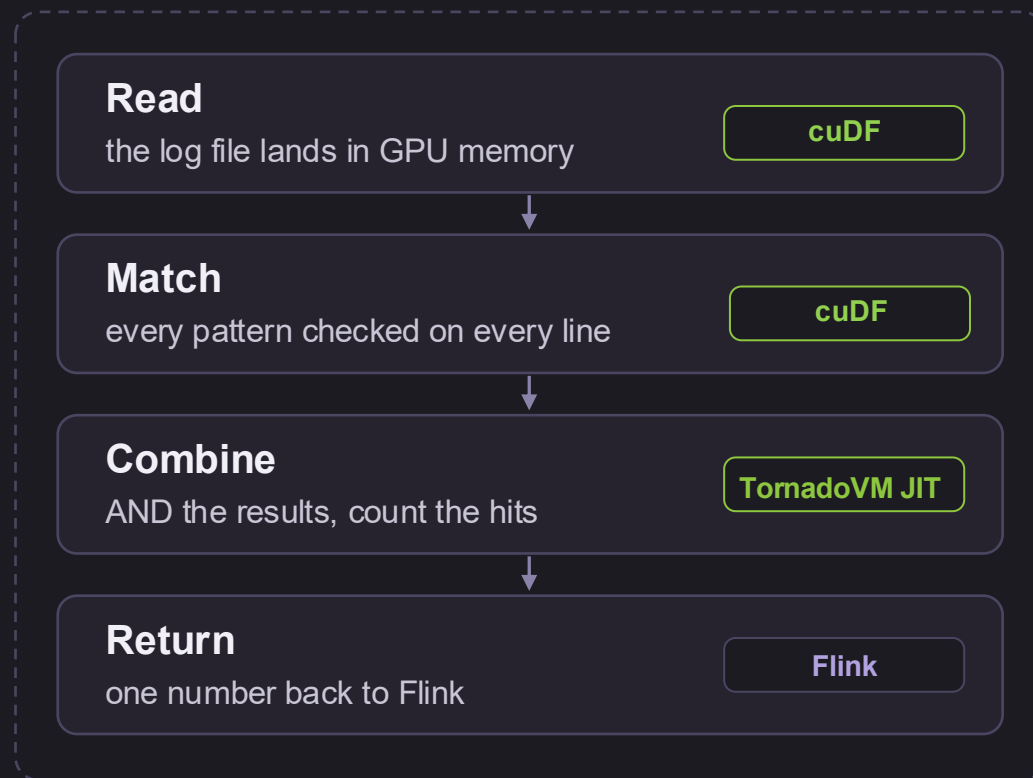
```
SELECT COUNT(*) FROM Logs
WHERE REGEXP(line, 'p1')
  AND REGEXP(line, 'p2')
  AND ...
  AND REGEXP(line, 'pk')
```

Plain Flink SQL: no hint, no annotation, no UDF.



planned and run
automatically

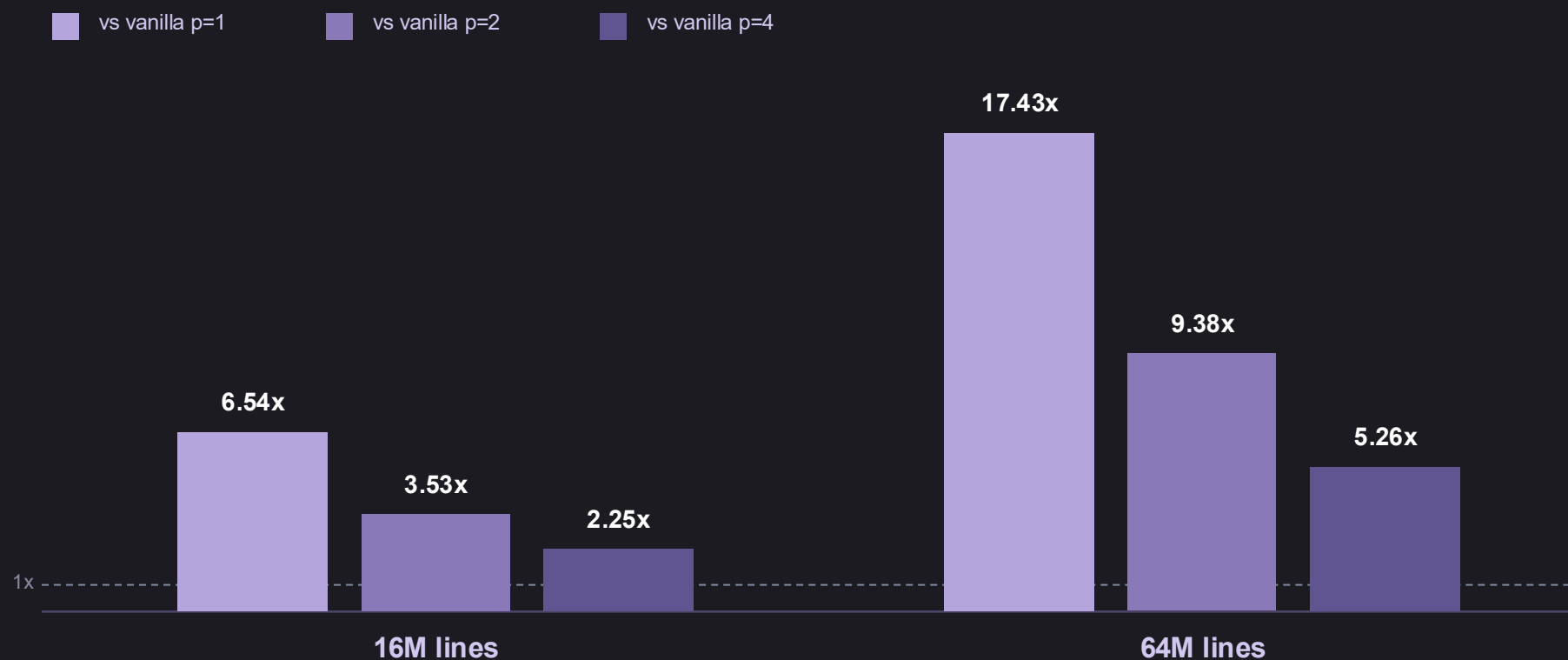
THE SYSTEM RUNS: ONE TASK GRAPH





Log screening: 17.4× on 64M lines

End-to-end wall clock of `./bin/flink run`, eight regexes per line · one GPU at parallelism 1 · vanilla Flink at 1, 2 and 4 slots.



Quadrupling the corpus barely moves the device arm — 6.0 s to 8.1 s — while vanilla goes from 39 s to 142 s. cuDF's regex engine is nowhere near saturated at 16M lines.

RTX 4070 Laptop · CUDA 12.5 · JDK 21 · Flink 2.3.0 · device median of 3 runs, vanilla mean of 2 · every run of both arms returned the full matched count.

Takeaway: Java frameworks can tap onto GPUs today!

Three demos, one platform: what to take back to your team.



JITLLM

1.12×

faster than llama.cpp on prefill

Qwen3-0.6B F16, pp512: 50,300 vs 44,805 tok/s.
Pure Java, cuBLAS for the prompt.



APACHE FLINK

17×

end to end, warm, on a Flink SQL query

For a regex query over 64M Parquet rows, combining
cuDF libraries with JIT compiled code.



APACHE Cassandra (JVector)

19×

faster graph build, ada002-1M

268.4 s on the CPU, 14.1 s on the GPU, recall@10
equal or higher. cuVS inside TornadoVM.



CUDA

98%

of CUDA C++ speed, plain Java kernels

Java kernels, CUDA Tile and NVIDIA libraries in one
CUDA graph. No JNI, no C++.

Quick install TornadoVM 7.1.0

Three ways to get started, with CUDA as the first-class NVIDIA backend.

1 Through SDKMAN!

Recommended

```
$ sdk list tornadovm
$ sdk install tornadovm 7.1.0-jdk22plus-cuda
```

2 Download the SDK directly

```
https://www.tornadovm.org/downloads
```

3 Clone & build from source

```
$ git clone https://github.com/beehive-lab/TornadoVM.git
$ cd TornadoVM && make BACKEND=cuda
$ source setvars.sh
```

Available versions

7.1.0-jdk22plus-cuda

latest

7.1.0-jdk21-cuda

7.1.0-jdk22plus-openc1

7.1.0-jdk22plus-metal

Download TornadoVM SDK

7.1.0 · JDK 22+

CUDA

linux/x86_64

Download

```
// module-info.java
module my.app { requires tornado.api;
}
```

Thank you! Questions?

Resources to take home.

1

TornadoVM

github.com/beehive-lab/TornadoVM

2

jitLLM

github.com/beehive-lab/jitllm

3

Demos from this talk

github.com/mikepapadim/tornadovm-devox2026-cuda-demos

4

SDKMAN!

sdkman.io/sdks/tornadovm

5

APACHE FLINK

github.com/mairooni/flink-accelerator-tornadovm

Scan to get started



tornadovm.org

GET IN TOUCH

Michalis Papadimitriou

in/[michalis-papadimitriou](#)
X [@mikepapadim](#)
github.com/mikepapadim

Mary Xekalaki

in/[mary-xekalaki](#)
X [@MXekalaki](#)
github.com/mairooni

We value your feedback!

Rate this talk



DEVOXX™

BACKUP

Takeaway: Java GPU programming is here on Nvidia HW

01

Keep your JDK

Any JDK 21–27: Temurin, Corretto, Zulu, GraalVM, OpenJDK

```
sdk install tornadovm
```

02

Offload one hot loop

@Parallel, KernelContext or TileContext on the method that dominates

```
.task()
```

03

Call NVIDIA's libraries

Where a vendor kernel exists, use it, on the same buffers

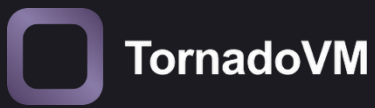
```
.libraryTask()
```

04

Remove host overhead

Capture once, replay; keep weights and state on the GPU

```
withCUDAGraph()
```



Thank you. Questions?

tornadovm.org · github.com/beehive-lab/TornadoVM
github.com/beehive-lab/GPULLama3.java
github.com/mikepapadim/tornadovm-devoxx2026-cuda-demos

Takeaways

What to take back to your team.

- 01 Kernels in plain Java** threads or tiles, compiled to inspectable CUDA within a few percent of hand-written code
- 02 NVIDIA libraries as graph nodes** cuBLAS, cuFFT, cuDNN, cuSPARSE, CUTLASS: no JNI, no bindings to maintain
- 03 One CUDA graph for everything** JIT, CUDA Tile and library nodes captured and replayed together
- 04 Each tool where it wins** libraries for compute-bound GEMM, your kernels for fused, memory-bound work
- 05 Incremental, on your JDK** one hot loop at a time, on any JDK 21–27

Try it today

SDKMAN, Maven Central and Docker images.

Linux, Windows and macOS.

Every demo has a hand-written CUDA twin to check the numbers against.

```
# JDK 22+ and the TornadoVM CUDA SDK (tile API)
sdk install java 25.0.2-open
sdk install tornadovm 7.0.0-jdk22plus-cuda
# CUDA 13.3 userspace for the CUDA Tile path
pip install --user nvidia-cuda-nvcc \
    'cuda-tile[tileiras]' nvidia-cuda-cccl

git clone https://github.com/mikepapadim/\
    tornadovm-devoxx2026-cuda-demos
cd tornadovm-devoxx2026-cuda-demos
source scripts/setup-env.sh
bash scripts/run-all-demos.sh
```



05

Use case 3

JVector + cuVS

Vector-index builds on the GPU: JVector's graph construction offloaded to NVIDIA cuVS through TornadoVM, with search and on-disk format unchanged.

JVector runs on GPU

Build, compaction and PQ run faster, and the graphs the GPU builds search faster at equal recall.

27×

graph build, ada002-1M, warm JVM

270.0 s on the CPU → 9.96 s on the GPU

15.2×

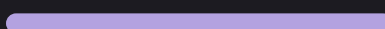
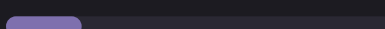
compaction, 4 segments

291.6 s → 19.2 s, and a better graph

−19%

p50 search latency, recall 0.98–0.99

on the GPU-built graph, 1 thread

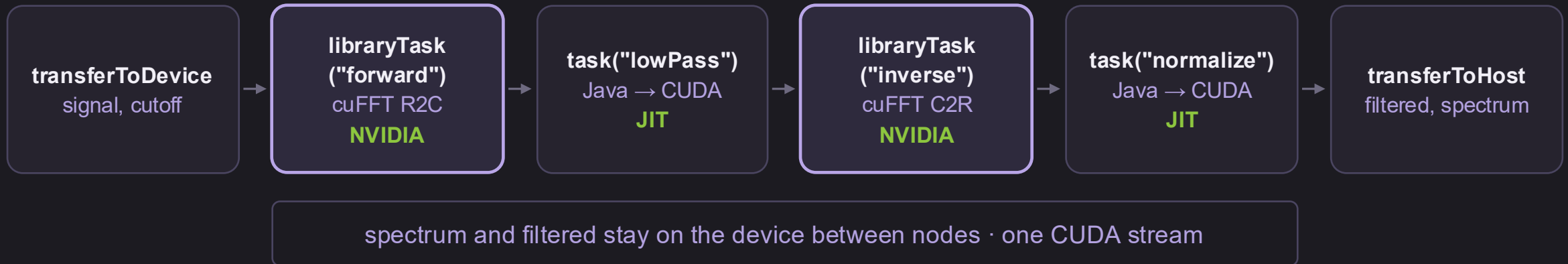
WIN	CPU	GPU	GAIN	QUALITY
Graph build · Cohere 10M ×1024	2270.7 s	207.1 s	 11×	recall@10 0.8752 → 0.8970
Graph build · ada002-1M, fresh JVM	267.4 s	14.3 s	 18.7×	recall@10 0.9599 → 0.9668
Compaction · ada002-1M, 4 segments	291.6 s	19.2 s	 15.2×	p50 −5% to −17%, equal recall
Bench build · on-disk, with writes	185.0 s	25.0 s	 7.4×	recall ≥ CPU at every overquery
PQ train + encode · M=192, K=256	13.56 s	3.80 s	 3.6×	identical codes, recall 0.8037 → 0.8071

SEARCH, EQUAL RECALL p50 −5% to −19% · QPS +3% to +14% at 32 threads · QPS +15% to +44% vs PQ-built graphs (Bench)

RTX 4090 (24 GB) vs i9-13900K, 32 threads, JDK 25 · M=32, efConstruction=100, alpha 1.2, hierarchy · equal recall: CPU curve interpolated at the GPU graph's recall

One task graph, two kinds of node

The live demo: a low-pass filter from two NVIDIA cuFFT calls and two Java kernels.



Same buffer model

signal, spectrum and filtered are FloatArrays: off-heap segments cuFFT reads and writes directly.

Checked every frame

The cutoff sweeps down at ~10 frames/s. Every run is compared with the exact answer, and the profiler reports each task's GPU time.

Same CUDA graph

Library nodes are capturable: withCUDAGraph() replays all four nodes as one launch, timed against plain execute().